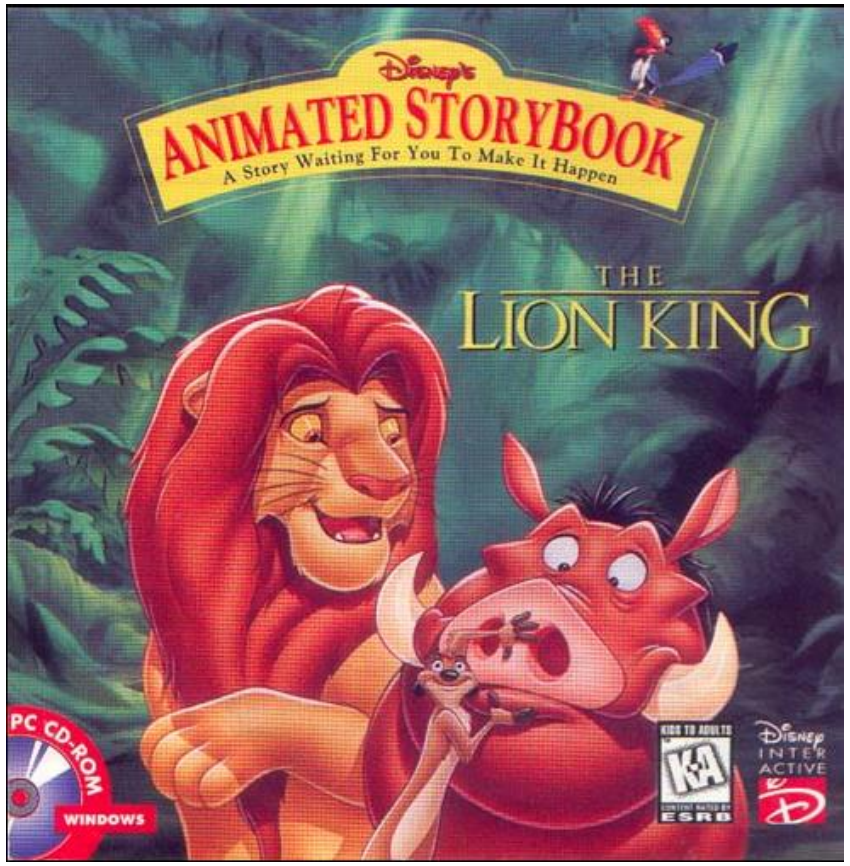
Chapter 6
Software Testing and Code Quality Assurance

Software Testing and Code Quality Assurance

1. Definition and classification of software testing
2. Definition of test cases and their design methodology
3. White Box Testing (Definition, Control Flow Chart, Logical Overlay Method, Test Case Design)
4. Black Box Testing (Definition, Equivalent Class Division, Boundary Value Analysis, Scenario Method)
5. Stress test, performance test and code coverage test
6. Code quality assurance

Typical case analysis of software defects



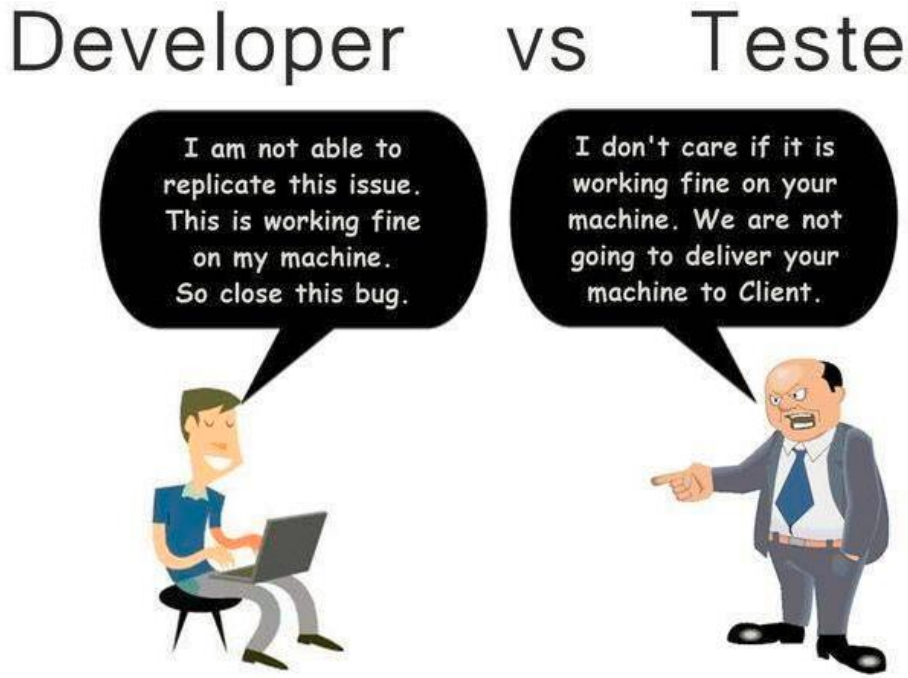
The Lion King Animated Storybook

Gmail

Boeing 737-800MAX

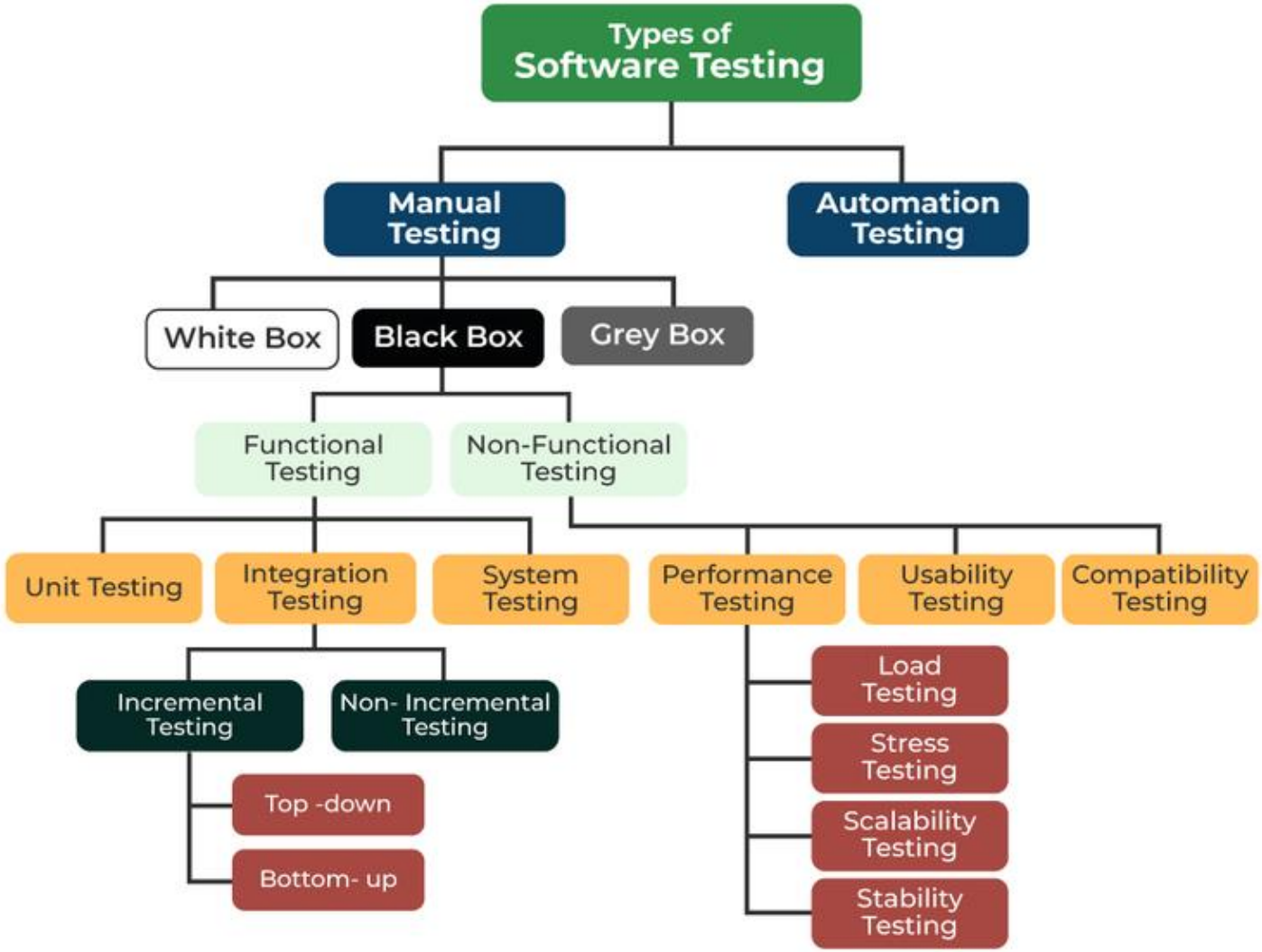
Definition of Software Testing (IEEE)

The process of running or measuring a system under test or statically checking a system under test using manual or automated means is designed to verify that the system under test meets the requirements and to find the difference between the actual system output and the expected results. Software testing is requirements-centric, not defect-centric!



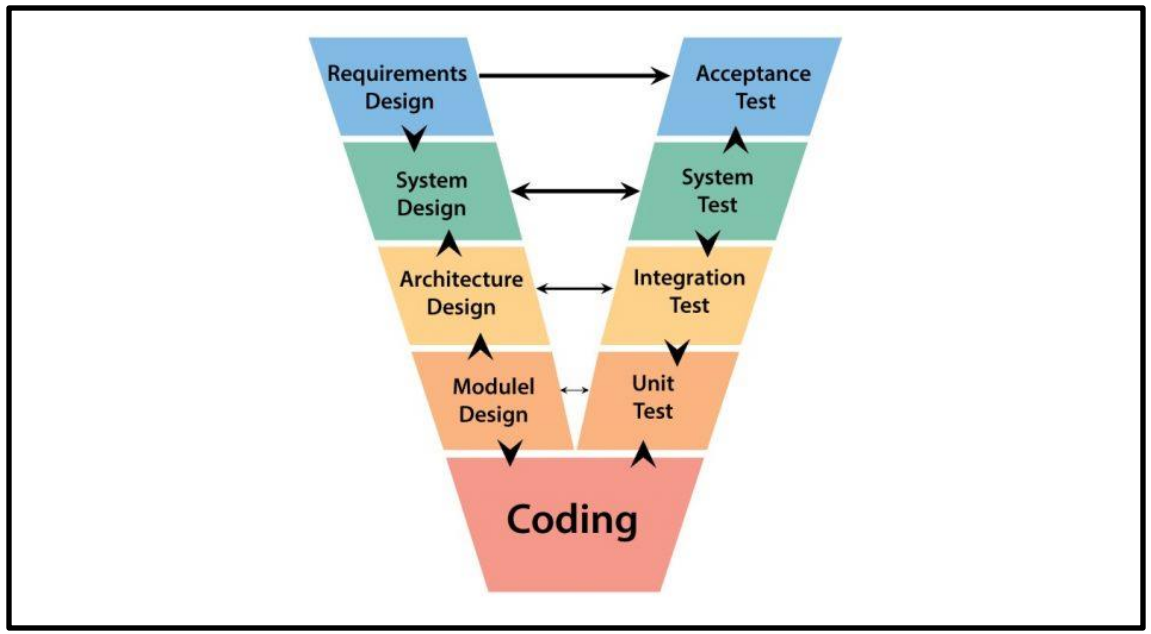
Classification of software testing

Different types or levels of software testing are performed during the software lifecycle, and a combination of these different types of tests is applied to ensure that the software system functions and behaves as expected.

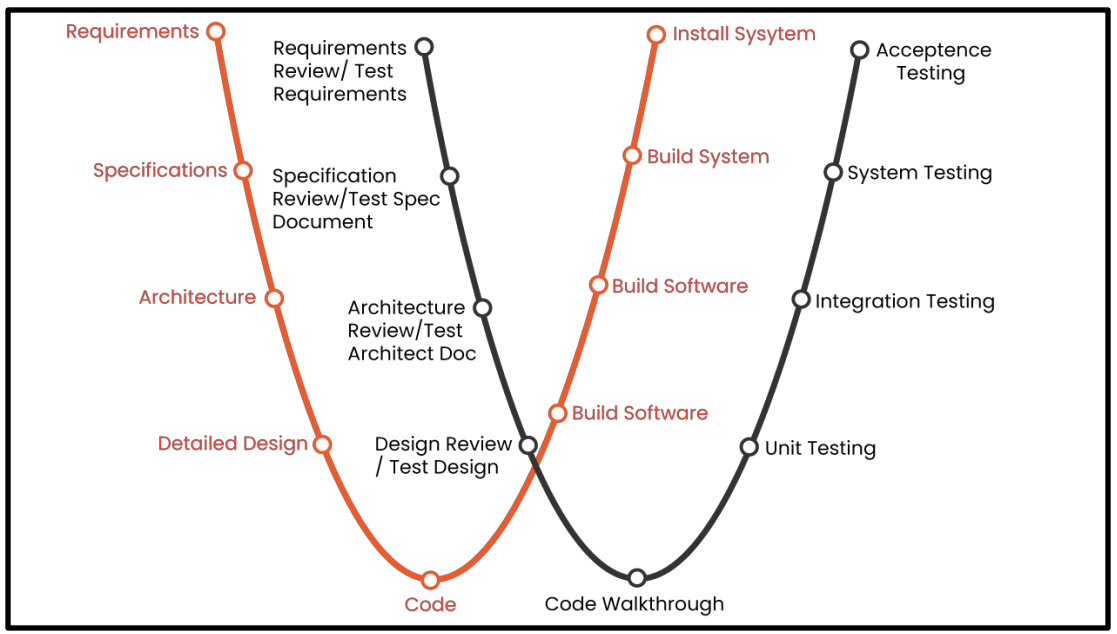


Classification of software testing

V model



W model



- Unit tests correspond to the coding stage, and their test objects are a single module or component.
- Integration testing corresponds to a detailed design, and its test object is a set of modules or components;
- System testing and acceptance testing correspond to the outline design and requirements phases, respectively, and are tested for the entire system.

Test Cases: Definitions

A test case is a description of a specific software product that performs a test task, reflecting the test scheme, method, technology, and strategy. Its content includes test objectives, test environment, input data, test steps, expected results, test scripts, etc., and finally form a document.

Simply think of a test case as a set of test inputs, execution conditions, and expected results compiled for a specific goal to verify that a specific software requirement is met.

Test Cases: Design Methodology

Test case is a scientific organizational summary of software testing behavior, with the aim of transforming software testing behavior into a manageable pattern. At the same time, test cases are also one of the ways to quantify the specific test, and test cases are different for different types of software.

The design methods of test cases mainly include black box testing and white box testing.

Black box testing, also known as functional testing, focuses on the external structure of the program, does not consider the internal logical structure, and mainly tests the software interface and software functions.

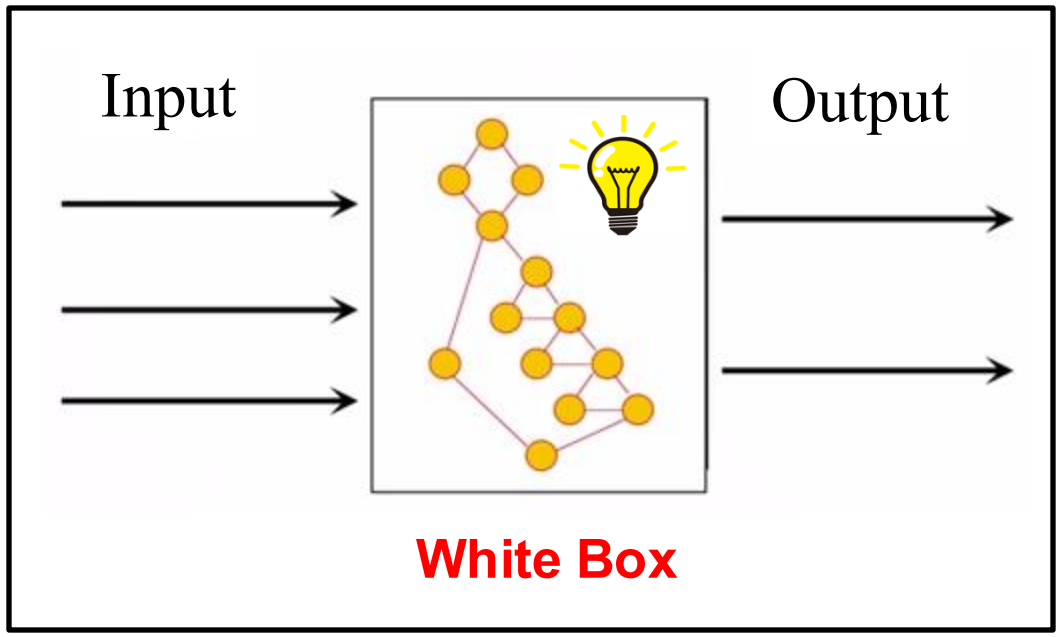
White box testing is also known as structural testing, transparent box testing, logic-driven testing, or code-based testing. The white box method comprehensively understands the internal logical structure of the program and tests all logical paths.

Test Cases: Design Principles

- Test case design generally follows the following principles：
  - Correctness. Input the actual data of the user to verify whether the system meets the requirements of the required specification. The test points in the test case should first ensure that they cover at least the features in the requirements specification and are normal.
  - Comprehensive. Cover all required functional items; In addition to testing the test point itself, the designed use cases also need to consider the actual use of users, the use of other parts, abnormal situations (unreasonable, illegal, cross-border and limit input data), operation and environment settings, etc.
  - Coherence. Use cases are organized and prioritized, especially in business test cases; Use case execution granularity. Try to keep each use case with a hit point, and not cover many functional points at the same time, otherwise the execution will be too implicated, so it is important to maintain coherence between each use case.
  - Determinable. The correctness of test execution results is determinable, and each test case has a corresponding expected result.
  - Operable. The test case should clearly write the test steps and the test results corresponding to the different operation steps.

White Box Testing: Definition

Also known as structural testing or logic-driven testing, it is based on the source code of the program, the internal working process of the known product, mainly to test the internal structure of the program, pay attention to the details of the program implementation, and check whether each channel in the program is working correctly according to the predetermined requirements.



advantage：

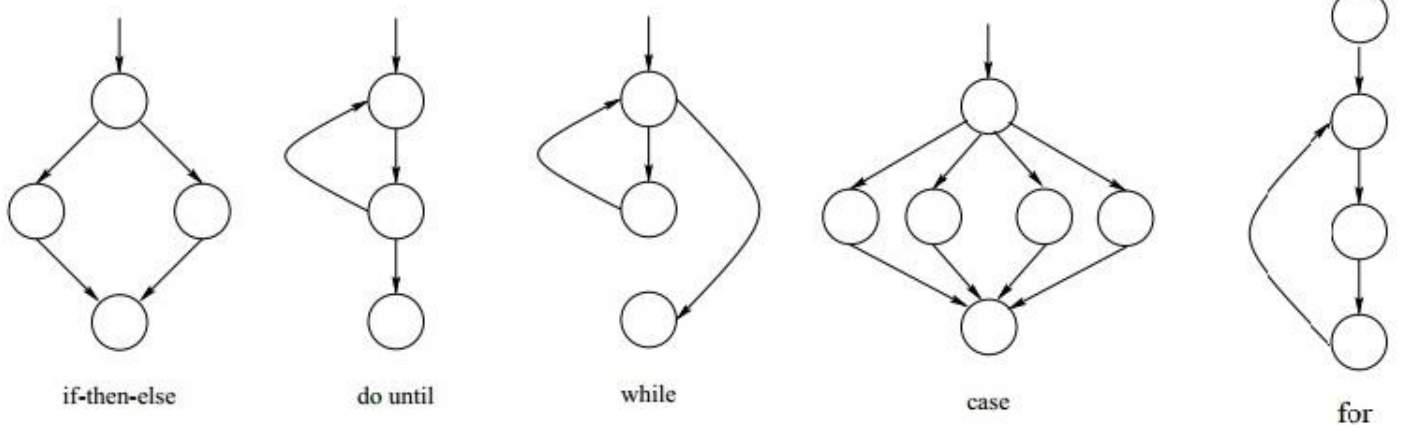
- Highly targeted and can quickly locate bugs
- At the function level, the cost of bug fixes is low
- It helps to understand how well the test is covered
- Helps optimize code and prevent defects

disadvantage：

- High requirements for testers
- High cost

White Box Testing: Control Flow Diagram

A graphical tool that represents the control structure of a program, and its basic elements are nodes, judgment nodes, and process blocks.



Defect risk: Linear < two-branched < multi-branched < loop structure

White box testing: Establish a model of the object under test

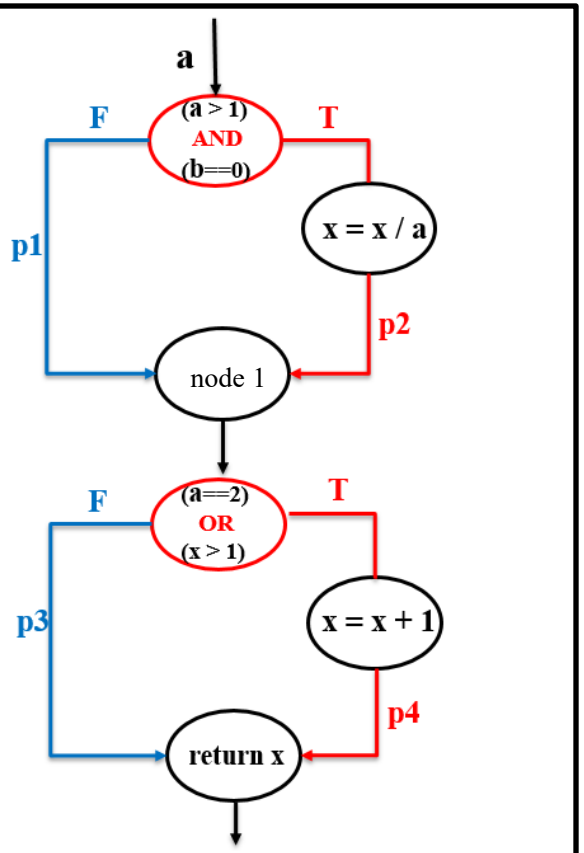
Source code

```
int Func1 (int a, int b, int x)
{
    if ((a > 1) && (b == 0))
        x = x / a;

    if ((a == 2) || (x > 1))
        x = x + 1;

    return x;
}
```

Control flow chart



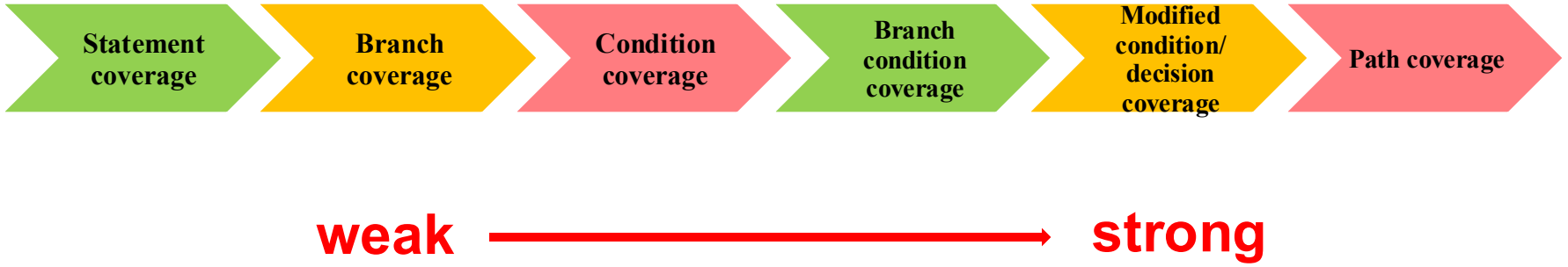
Statement v.s. condition v.s. branch

```
public class PassCheck {
    public static void main(String[] args) {
        int score = 75;
        if (score >= 60) {
            System.out.println("Pass");
        } else {
            System.out.println("Fail");
        }
    }
}
```

White Box Testing: A logical override method

Tests for judgments

- Focus: Determine the complexity of the expression itself
- Principle: Examine the judgment expressions in the source code, and realize the coverage of the program by traversing the logical structure of the program.
- Common coverage metrics:



White Box Testing: A logical override method

Coverage metrics	
Statement coverage	Execute all statements in the program at least once. Statement override is the weakest logical override criterion.
Branch coverage	Also known as branch override, each branch in the program is executed at least once, ensuring that each decision node in the program achieves each possible result at least once.
Condition coverage	In each compound judgment expression in the guarantee program, the truth and falsehood of each simple judgment condition are executed at least once.
Branch condition coverage	Ensure that the true and false situations of each simple decision condition are executed at least once in each compound decision expression in the program (condition override), and ensure that each possible result is obtained at least once for each decision node in the program (decision override).
Modified condition/ decision coverage	All possible value combinations of all simple decision conditions are executed at least once in each decision node of the program.
Path coverage	Make every possible path in the program execute at least once. This strategy is the strongest, but generally unattainable.

White Box Testing: An Example

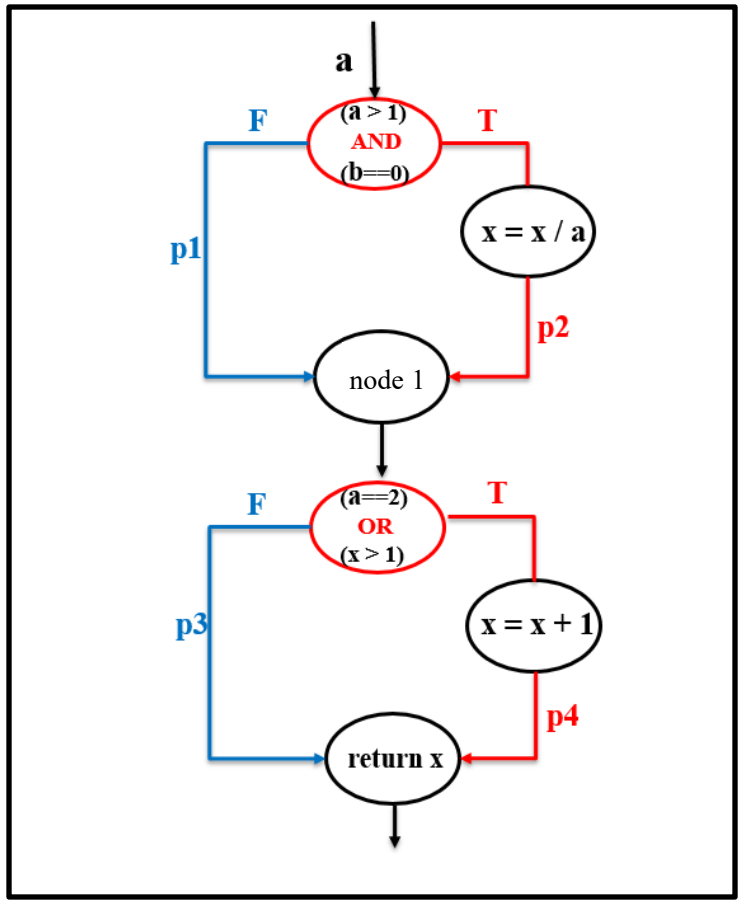
- **4 basic logical conditions**

T1	a > 1
T2	b==0
T3	a==2
T4	x>1

- **4 paths**

L13	p1->p3
L14	p1->p4
L23	p2->p3
L24	p2->p4

Control flow chart



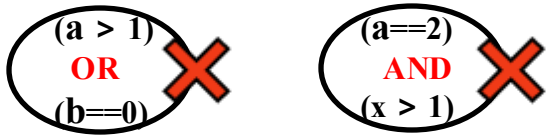
White Box Testing: An Example

1. Statement coverage

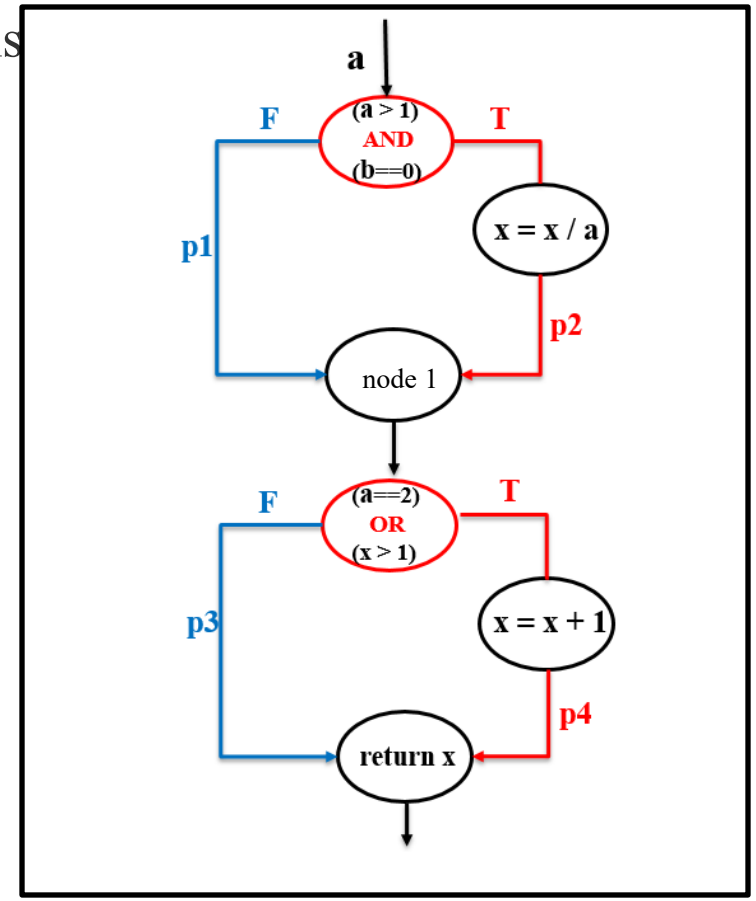
Execute all statements in the program at least once. Statement coverage is weakest criterion.

Test case ID	input			anticipate output	cover path
	a	b	x	x	
TC1	2	0	4	3	L24

Missing cases:



Control flow chart



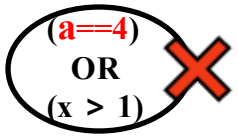
White Box Testing: An Example

2. Branch coverage

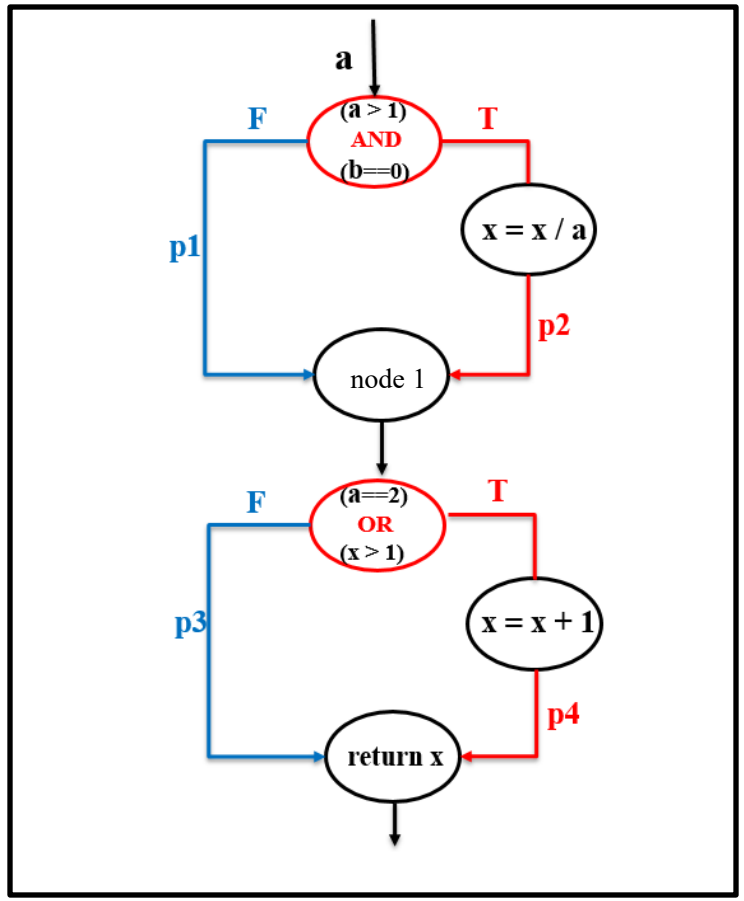
Each branch in the program is executed at least once, ensuring that each decision node in the program achieves each possible result at least once.

Test case ID	input			anticipate output	cover path
	a	b	x	x	
TC1	1	1	2	3	L14
TC2	3	0	3	1	L23

Missing cases:



Control flow chart



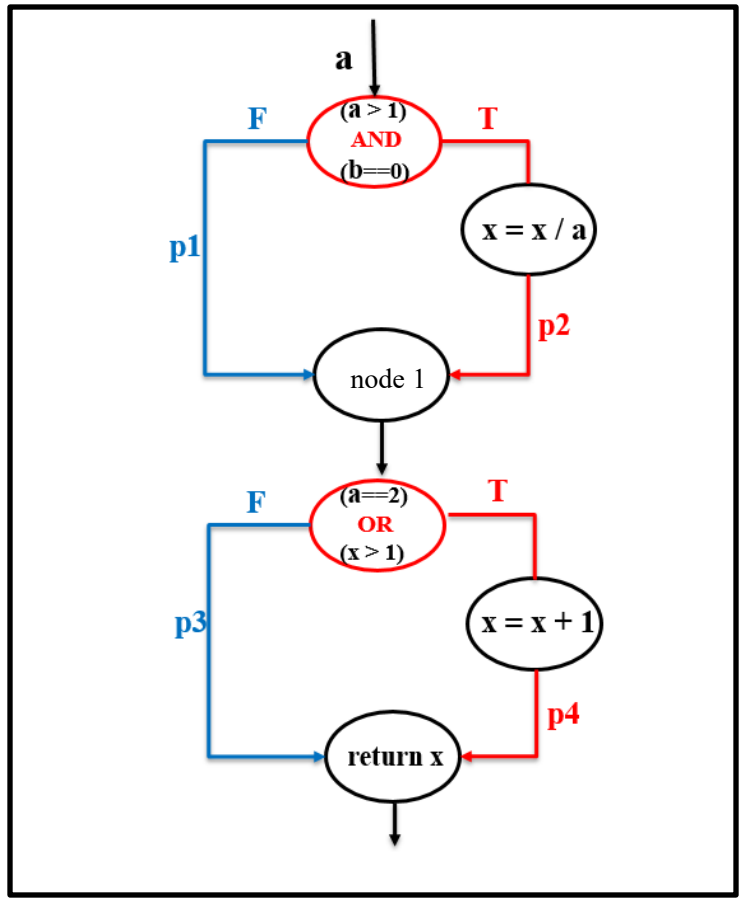
White Box Testing: An Example

3. Condition coverage

Not only is each statement executed at least once, but each condition in the decision expression is made to get various possible results (true or false).

Test case ID	input			anticipate output	cover path	cover condition
	a	b	x	x		
TC1	2	1	0	1	L14	T1(T) T2(F) T3(T) T4(F)
TC2	1	0	2	3	L14	T1(F) T2(T) T3(F) T4(T)

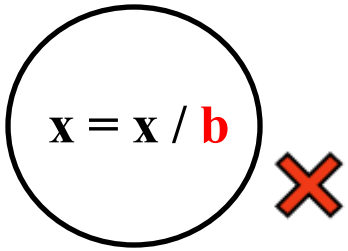
Control flow chart



- 4 basic logical conditions

T1	a > 1	T3	a==2
T2	b==0	T4	x > 1

Missing cases:



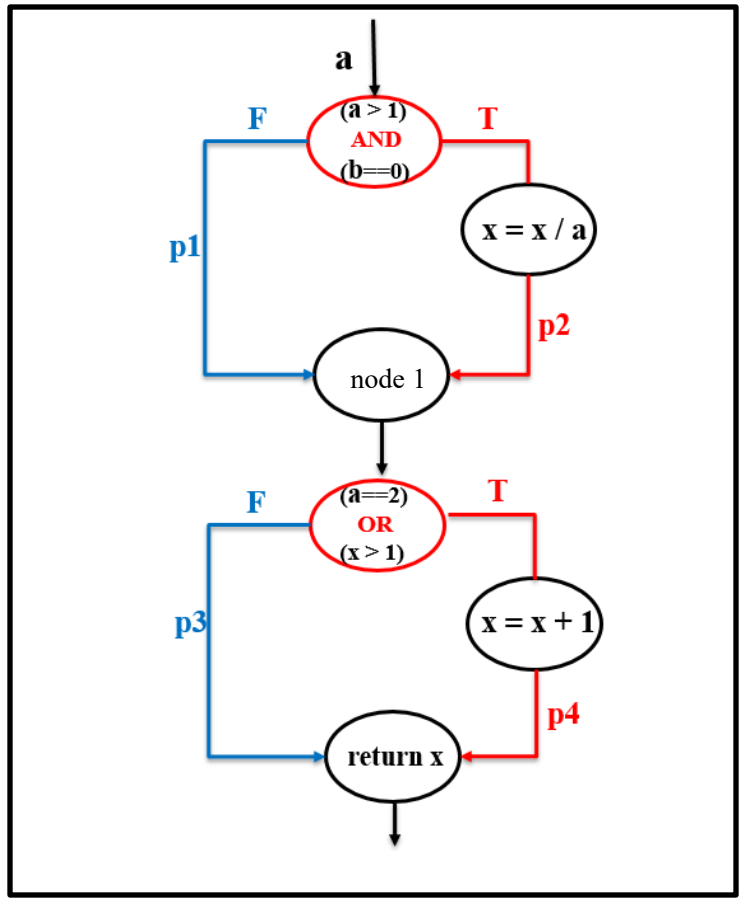
White Box Testing: An Example

4. Branch condition coverage

Both the branch coverage and condition coverage are met.

Test case ID	input			anticipate output	cover path	cover condition
	a	b	x	x		
TC1	2	0	4	3	L24	T1(T) T2(T) T3(T) T4(T)
TC2	1	1	1	1	L13	T1(F) T2(F) T3(F) T4(F)

Control flow chart



- 4 basic logical conditions

T1	a > 1	T3	a==2
T2	b==0	T4	x > 1

Missing cases: (a==2) OR (x < 1)

White Box Testing: An Example

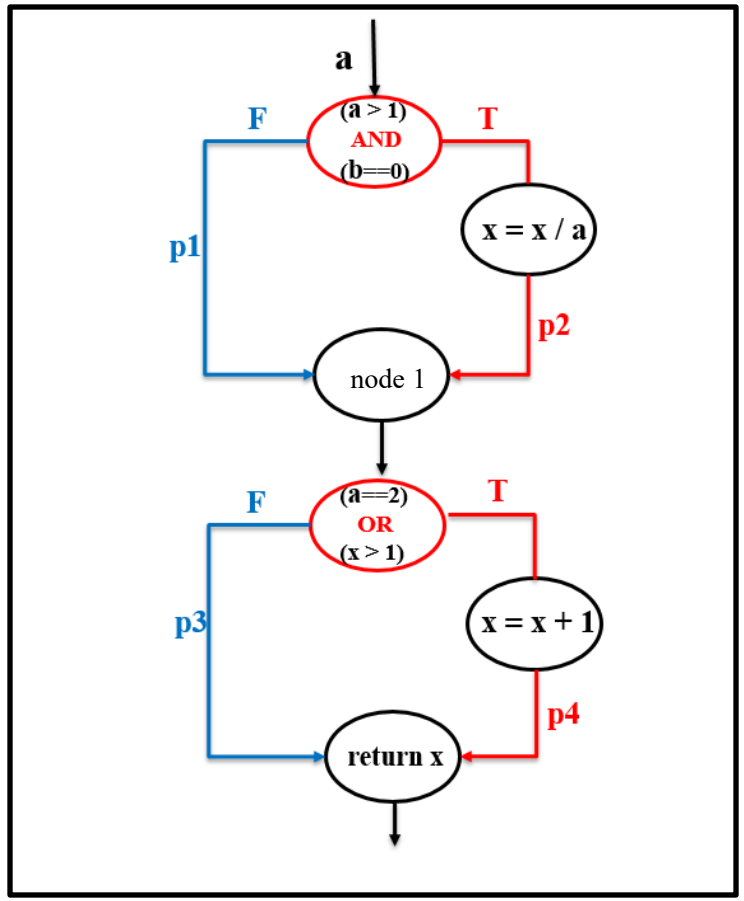
5. Modified condition/decision coverage

All possible value combinations of all conditions are executed at least once in each decision node of the program.

Conditions	
T1:a > 1	T2:b==0
T	T
T	F
F	T
F	F

Conditions	
T3:a==2	T4:x > 1
T	T
T	F
F	T
F	F

Control flow chart



White Box Testing: An Example

5. Modified condition/decision coverage

Design use cases to achieve modified condition/ decision coverage :

TC1, TC6, TC9, TC12

disadvantages:

- Redundancy may exist. For example, the use cases in the red box have the same override path.
- The number of combinations is large and it takes a lot of time

Test case ID	input			anticipate output	cover path				cover condition
	a	b	x	x	T1	T2	T3	T4	
TC1	2	0	4	3	T	T	T	T	L24
TC2	2	0	2	2	T	T	T	F	L24
TC3	3	0	6	3	T	T	F	T	L24
TC4	3	0	3	1	T	T	F	F	L23
TC5	2	1	2	3	T	F	T	T	L14
TC6	2	1	1	2	T	F	T	F	L14
TC7	3	1	2	3	T	F	F	T	L14
TC8	3	1	2	2	T	F	F	F	L13
TC9	1	0	2	3	F	T	F	T	L14
TC10	1	0	1	1	F	T	F	F	L13
TC11	1	1	2	3	F	F	F	T	L14
TC12	1	1	1	1	F	F	F	F	L13

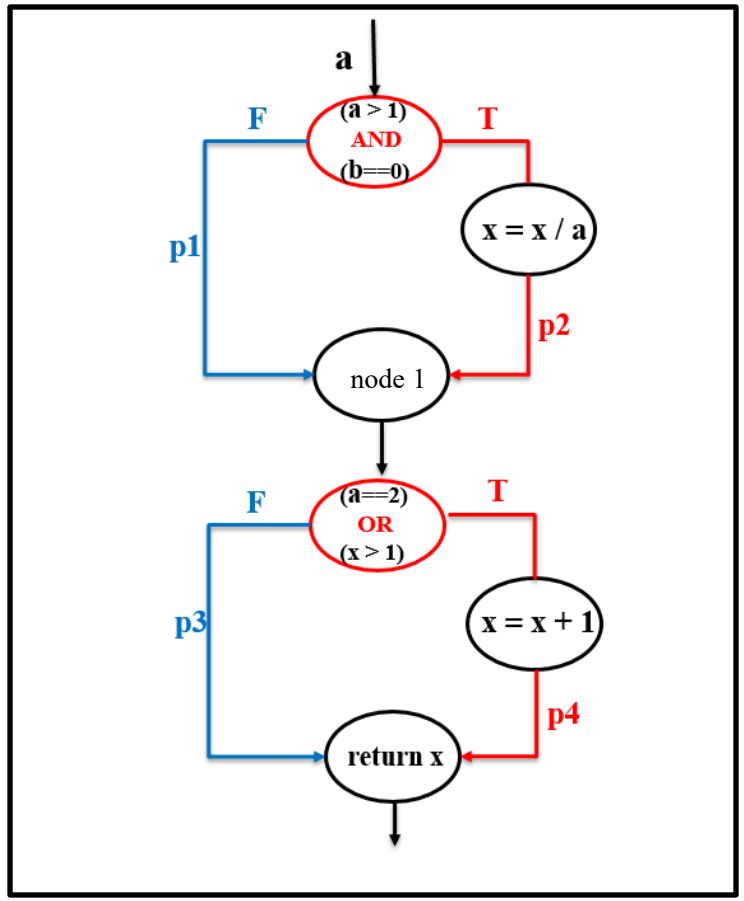
White Box Testing: An Example

6. Path coverage

Make every possible path in the program execute at least once. This strategy is the strongest, but generally unattainable.

Test case ID	input			anticipate output	cover path
	a	b	x	x	
TC1	2	0	4	3	L24
TC2	1	1	1	1	L13
TC3	1	1	2	3	L14
TC4	3	0	3	1	L23

Control flow chart

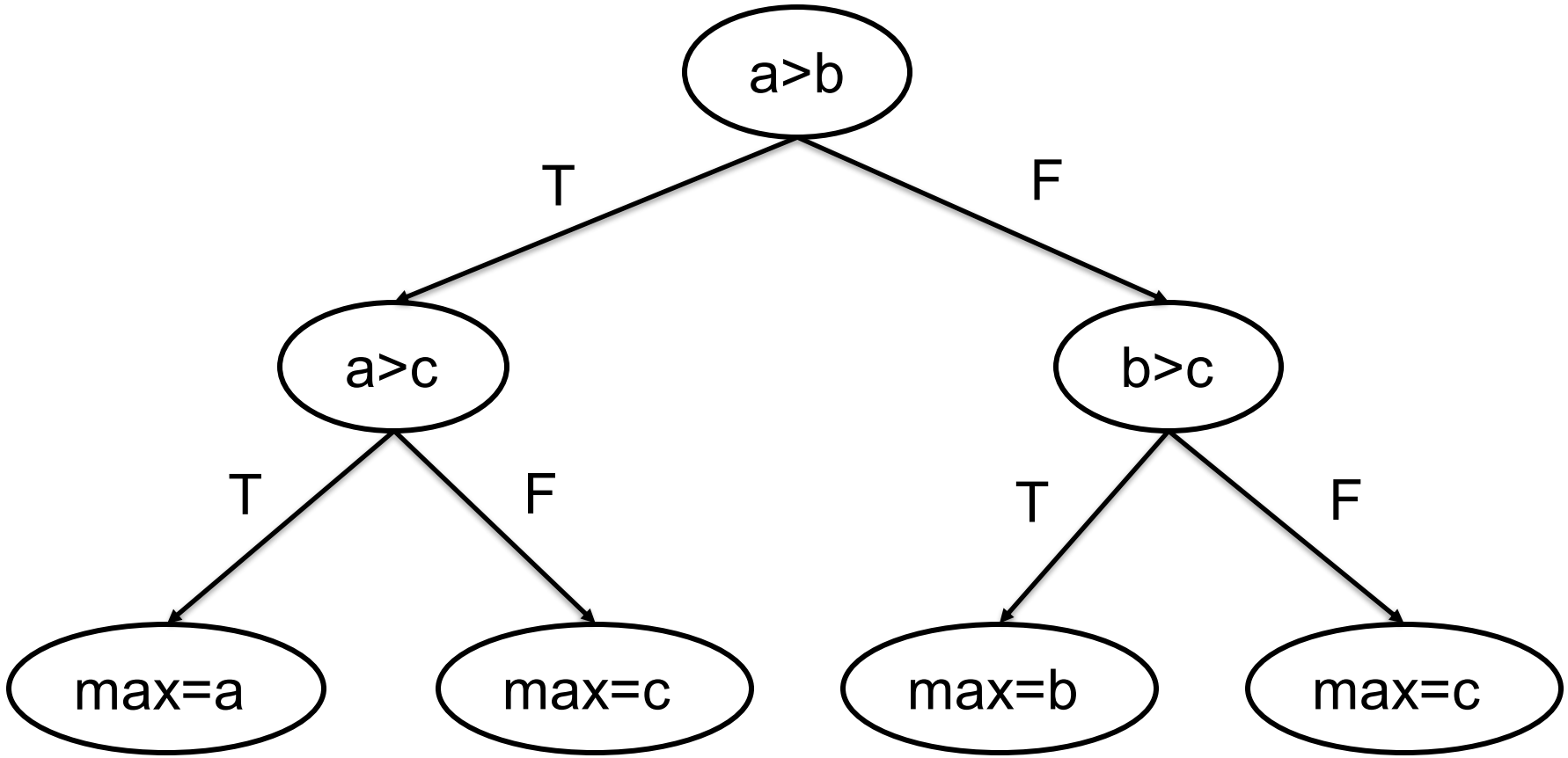


Quiz

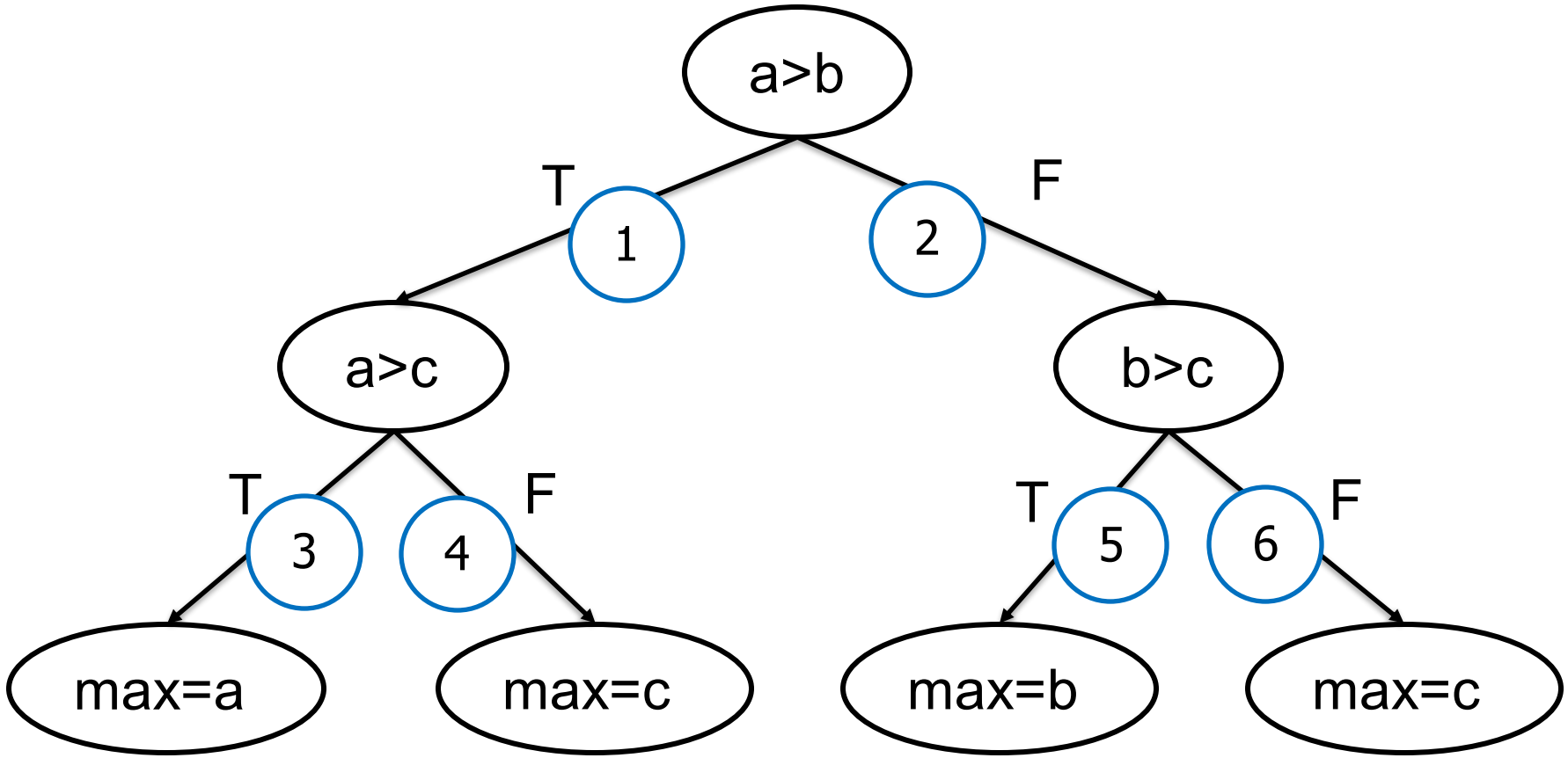
Plot the control flow diagram and design test cases to achieve branch coverage.

```java
public class MaxFinder {
    public static int findMax(int a, int b, int c) {
        int max;
        if (a > b) {                    // Decision 1
            if (a > c)                  // Decision 2
                max = a;
            else
                max = c;
        } else {
            if (b > c)                  // Decision 3
                max = b;
            else
                max = c;
        }
        return max;
    }
}
```

Quiz



Quiz

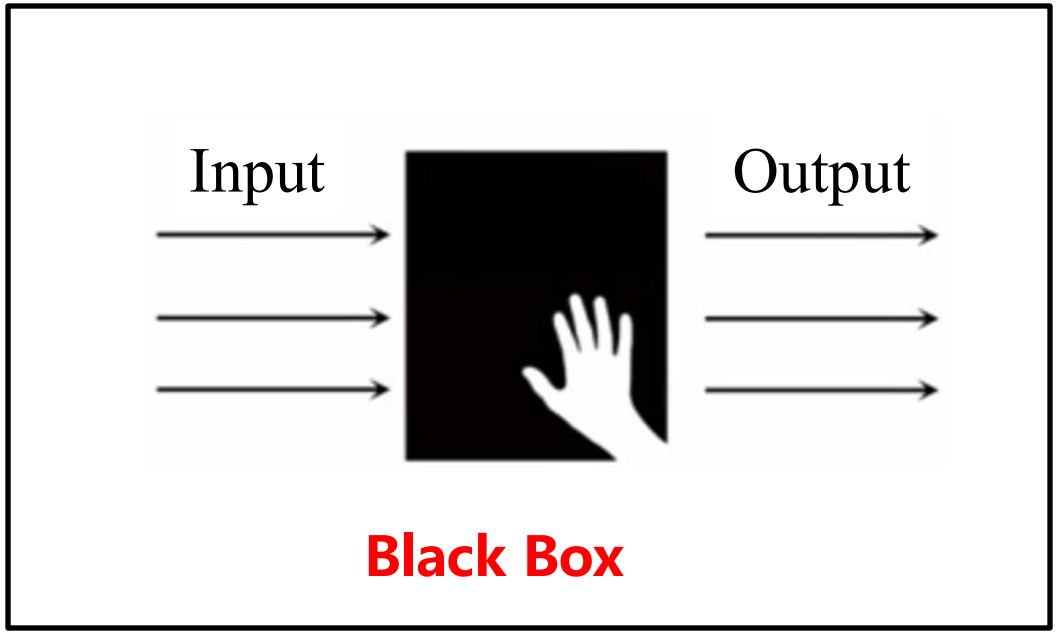


Quiz

Test Case	a	b	c	Expected Output	Covered Conditions	Covered Branches
1	5	3	1	5	Condition 1 (T), Condition 2 (T)	13
2	5	3	7	7	Condition 1 (T), Condition 2 (F)	14
3	3	5	1	5	Condition 1 (F), Condition 3 (T)	25
4	3	5	9	9	Condition 1 (F), Condition 3 (F)	26

Black Box Testing: Definition

Also known as functional testing or data-driven testing, it focuses on the external structure of the program, without considering the internal logical structure and internal characteristics of the program, the tester tests on the program interface, it only checks whether the program functions are used normally according to the requirements specifications, and whether the program can properly receive input data and produce correct output information.



advantage：

- The method is simple and effective
- The system behavior can be tested as a whole
- Development and testing can go hand in hand
- The technical requirements for testers are relatively low

disadvantage：

- Low coverage
- Directly dependent on the requirements specification
- The entry threshold is low

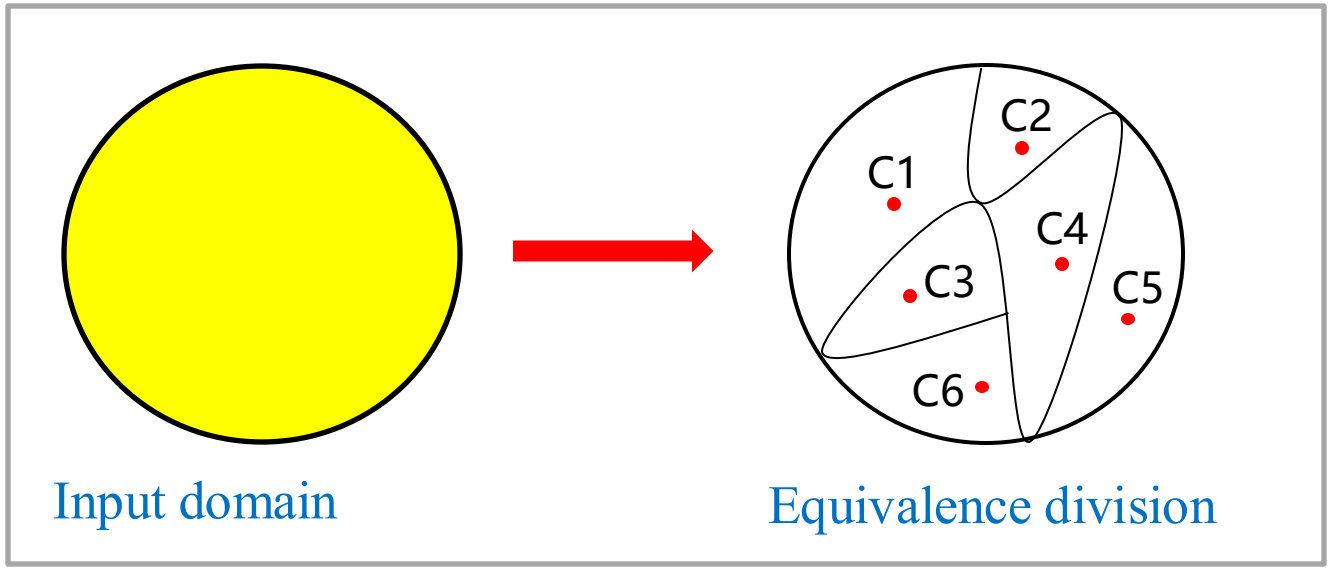
Black box testing: equivalence classification

Equivalent class

A subset of the input field in which the individual input data pairs are equivalent to errors in the revealing program.

Equivalence tests

Reduce infinite data to a limited number of equivalent regions and complete exhaustive testing by testing equivalent regions.



- Divide but do not interact
- Combine without changing
- Intra-class equivalence

Black box testing: equivalence classification

Effective equivalence class

- It refers to the collection of input data that is reasonable and meaningful for the specification of the software.
- It is used to verify whether the system under test can complete the specified function.

Invalid equivalence class

- It refers to the collection of input data that is unreasonable and meaningless for the specification of the software.
- It is used to investigate the fault tolerance of the system under test.

Black box testing: equivalence classification

Division principle:

①If the input condition specifies a range or number of values, one valid equivalence class and two invalid equivalence classes can be determined. (e.g. quantity from 1 to 999)

②If the input condition specifies a set of input values (assuming n), and the software treats each input differently, it can determine n valid and one invalid equivalent class.

③If there is a condition where the input condition states "must be", a valid equivalent class and an invalid equivalent class can be determined. (e.g., the first character of the identifier must be a letter)

④If you enter a Boolean expression, you can determine a valid equivalent class and an invalid equivalent class.

Black Box Testing: An Example

The query conditions of the system are from January 1990 to December 2019, which are represented by 6 digits, with the first 4 digits indicating the year and the last 2 digits indicating the month.

- **Determine valid and invalid equivalence classes:**

Criteria	Effective equivalence class		Invalid equivalence class	
Type and length of dates	**C1**	6-digit characters	**C2**	non-numeric characters
			C3	Less than 6 digit characters
			C4	More than 6 digit characters
Year range	**C5**	1990~2019	**C6**	Less than 1990
			C7	Greater than 2019
Month range	**C8**	01~12	**C9**	equal to 00
			C10	Greater than 12

Black Box Testing: An Example

- **Write test cases that cover all equivalent classes**
  - Covers effective equivalence classes

Test data	Desired results	Valid equivalent class
201408	**valid**	**C1、C5、C8**

  - Coverage of invalid equivalence (single defect principle)

Test data	Desired results	Invalid equivalent class
aaa123	invalid	**C2**
2011	invalid	**C3**
20140408	invalid	**C4**
187905	invalid	**C6**
209811	invalid	**C7**
200100	invalid	**C9**
200156	invalid	**C10**

Black Box Testing: Boundary value analysis

It is a commonly used black box testing technique. It tends to select data from or near the system boundary to design test cases, and test cases that take boundary conditions into account have a higher return on test.

Basic principle

Long-term testing experience shows that a large number of defects often occur at the boundaries of the input or output domain. Therefore, some test cases are designed to run the program near the boundary situation, so that the possibility of exposing errors is higher.

Black Box Testing: Boundary value analysis

Boundary point:

Refers to the points in the equivalence class that happen to be at the boundary and may cause changes in the internal processing mechanism of the system under test.

- example

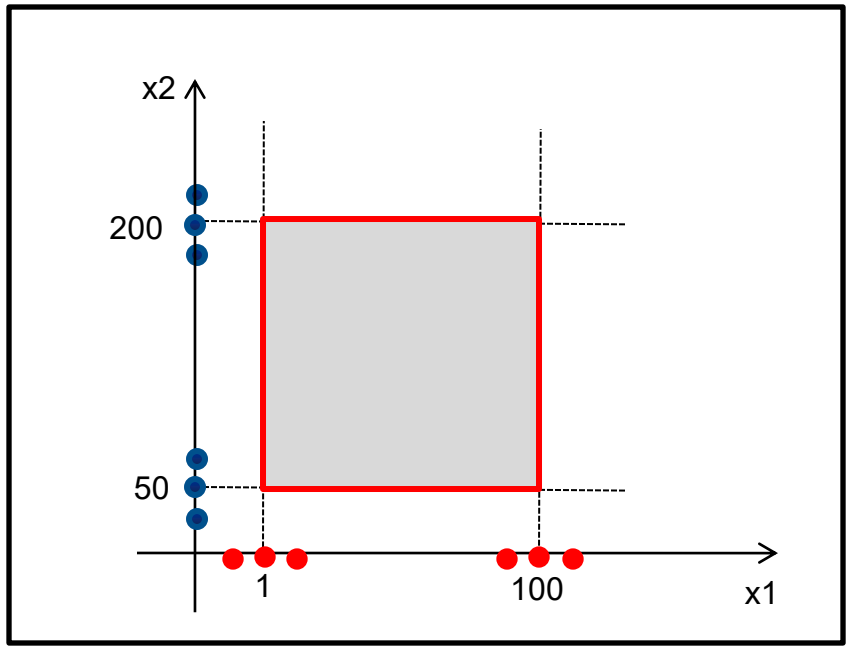
int Add (int x1, int x2)

1 <= x1<=100

50<= x2<=200

- Description of requirements:

- For valid inputs, the function returns the sum of x1 and x2;
- For invalid inputs, the function returns -1.



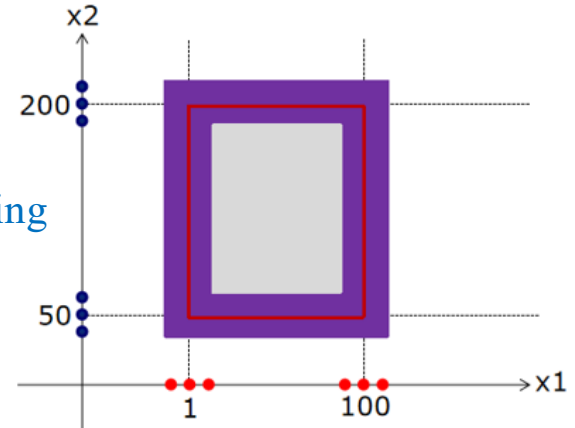
Locate the boundary point for each input condition to find the system boundary.

Black Box Testing: Boundary value analysis

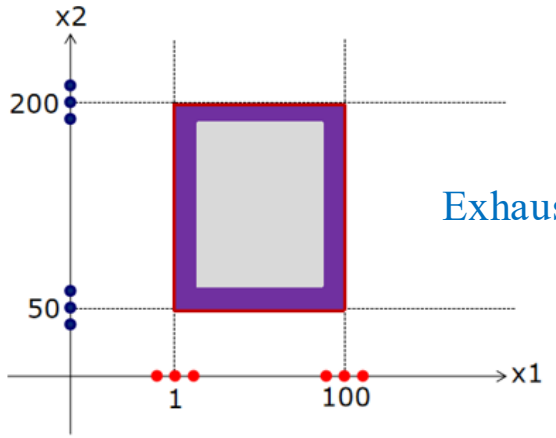
Organize test data design test cases:

Use case evaluation criteria: quantity, coverage, redundancy, defect localization ability, and complexity.

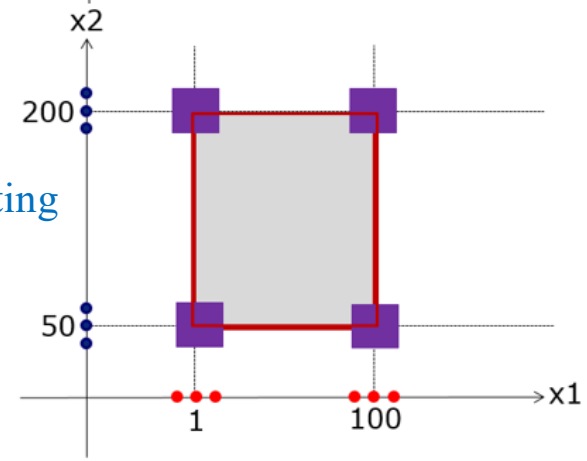
Exhaustive testing



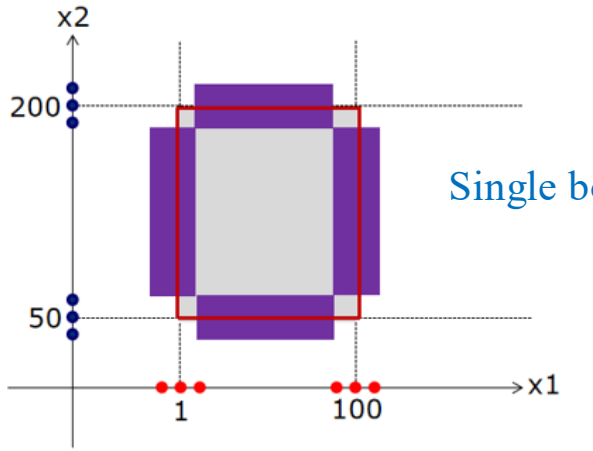
Exhaust effective domain testing



Multi-border testing



Single boundary test



Black Box Testing: Boundary value analysis

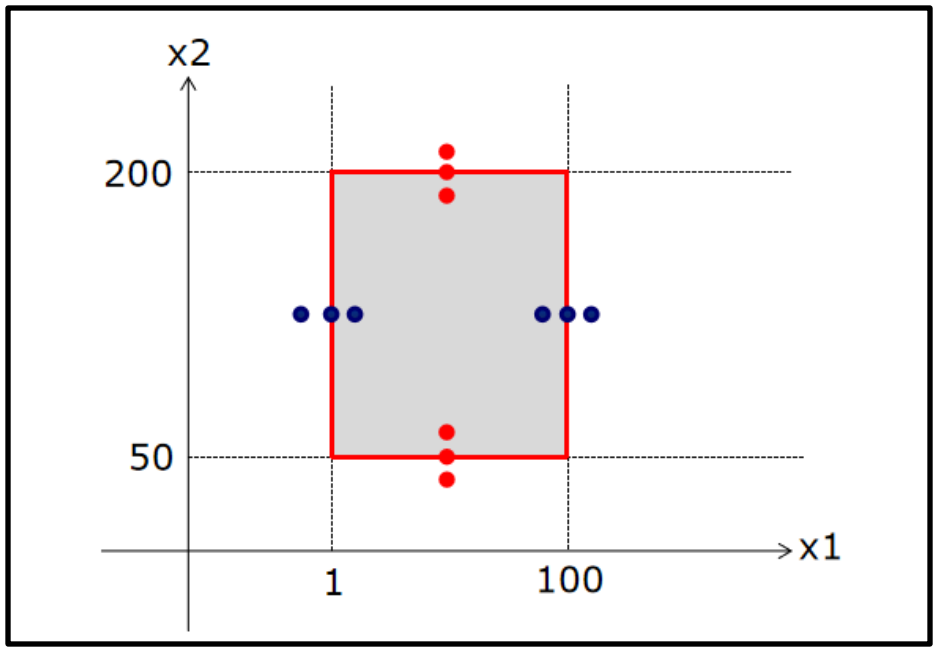
Organize test data design test cases:

- **Optimized single boundary**

int Add （int x1, int x2）

1 <= x1<=100

50<= x2<=200



- Number of use cases: 12
- Coverage: 100%
- Redundancy: None
- Defect localization ability: easy
- Complexity: Low

Black Box Testing: Scenario Method

With event flow as the core, the main purpose is to test the main business processes, main functions of the software, and the ability to handle exceptions.

Test steps：

- Define basic and alternative flows;
- Define the scenario;
- Design test cases from scenarios;
- Input test data to refine test cases.

Black Box Testing: An Example

Example: ATM withdrawals

Basic Flow (Correct Withdrawal)	
1	Insert a bank card
2	Verify the card
3	Enter your password
4	Verify the password
5	Go to the main interface of the ATM
6	Select the amount
7	ATM verification
8	Update account balance and withdraw cash
9	Return to the main interface

Alternative Flow (Wrong Withdrawal)	
1	Wrong bank card
2	Wrong password
3	The password was wrong 3 times
4	Insufficient balance on the card
5	Exceed the balance of the day
6	The ATM balance is insufficient

Black Box Testing: An Example

- Define scenarios and design test cases

Use case ID	scenario	Bank card	pass word	balance	Day limit	balance	Expected results
1	**Successful withdrawal**	**V**	**V**	**V**	**V**	**V**	**Successful withdrawal**
2	The bank card is invalid	I	N/A	N/A	N/A	N/A	Error, card refund
3	Wrong password	V	I	N/A	N/A	N/A	Error
4	Password is wrong 3 times	V	I	N/A	N/A	N/A	Error, card not refund
5	Insufficient account balance	V	V	I	N/A	N/A	Error, card refund
6	The total withdrawal amount exceeds day limit	V	V	V	I	N/A	Error, card refund
7	balance is insufficient	V	V	V	V	I	Error, card refund

Note: V indicates a valid value; I indicates invalid values; N/A indicates not applicable.

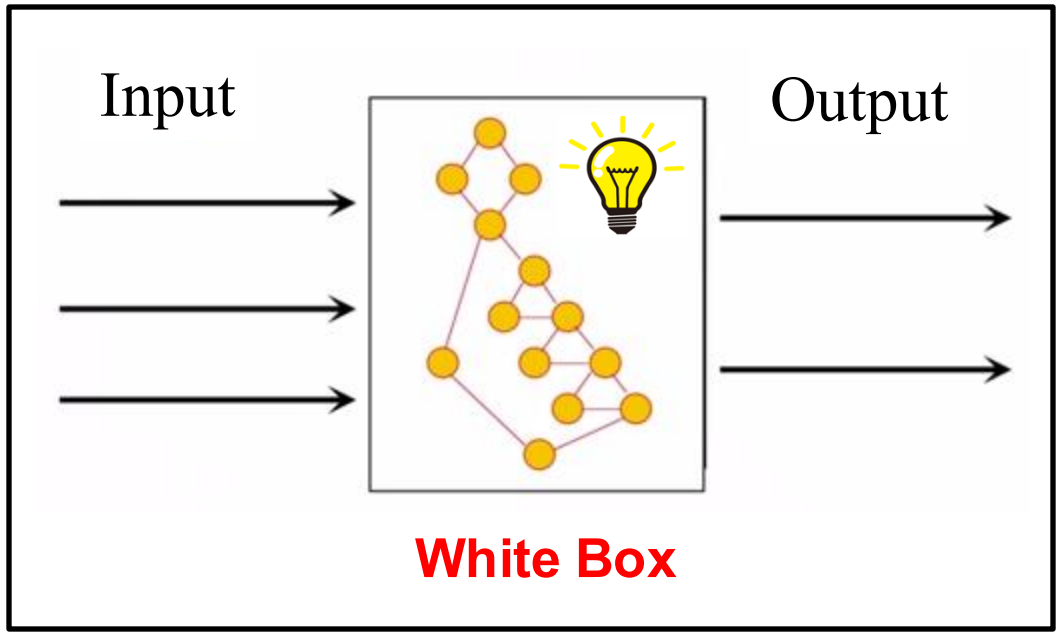
Black Box Testing: An Example

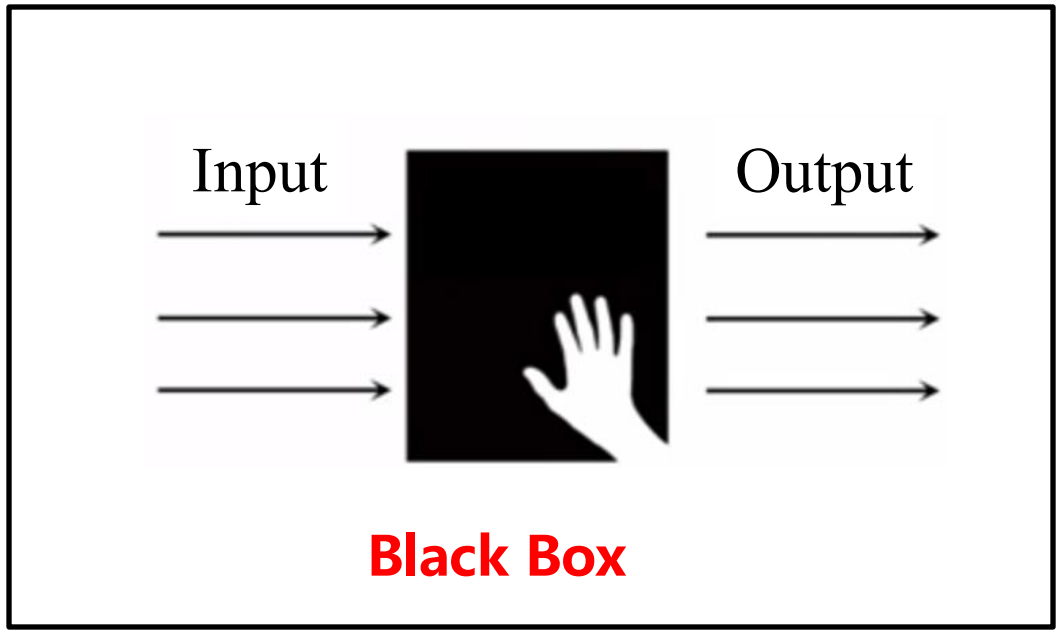
Use case ID	Scenario	Precondition	Procedure	Expected results
C1	**Scenario 1**	① ATM balance of 5,000 yuan ② Bank card number: 1234567812345678 ③ Password: 123456 ④ Card balance: 2,000 yuan	① Insert a bank card ② Enter the correct password ③ After entering the homepage, choose to withdraw 1,000 yuan	① The ATM machine outputs 1,000 yuan, return to the main interface ② ATM balance 4,000 yuan ③ The balance in the user's card is 1,000 yuan
C2	**Scenario 2**	ATM balance of 4,000 yuan	Insert an invalid bank card	Prompt the bank card to be invalid and return the card
C3	**Scenario 3**	① ATM balance of 4,000 yuan ② Bank card number: 1234567812345678 ③ Password: 123456 ④ Card balance: 1,000 yuan	① Insert a bank card ② Enter the wrong password: 654321	Prompt the wrong password and clear the password
C4	**Scenario 3**	① ATM balance of 4,000 yuan ② Bank card number: 1234567812345678 ③ Password: 123456 ④ Card balance: 1,000 yuan	Enter the wrong password again on the basis of C3: 987654	Prompt the wrong password and clear the password
C5	**Scenario 4**	① ATM balance of 4,000 yuan ② Bank card number: 1234567812345678 ③ Password: 123456 ④ Card balance: 1,000 yuan	Enter the wrong password again on C4: 876543	Indicates an incorrect password and confiscates the card

Black Box Testing

Comparison

- White Box Test (structure Test): mainly detects errors in the software coding process.
- Black box test (functional test): mainly to test whether the function can be used normally according to the specification.





- High requirements for testers
- Discover the problem in the code

- Relatively simple
- Test features from the user's perspective

Unit Testing

It is a code-level test that checks and verifies the smallest testable unit in the software. Unit testing, as the name suggests, is to test a "unit", which is different from integration testing, which is generally a class or function, not a module or system.

Unit tests can be seen as part of the coding work and should be completed by programmers.

Unit tests and JUnit

JUnit: A unit testing framework for the Java language

- Built by Kent Beck (Extreme Programming) and Erich Gamma (Design Patterns).
- It is the most successful of the xUnit family.
- Most Java IDEs integrate JUnit as a unit testing tool.
- Official website: http://www.junit.org

Better tools make good work!

Unit tests and JUnit

Traditional testing method: Use the main function

*Calculator.java

```
1  package com.junit.main;
2
3  public class Calculator {
4
5      public int add(int a, int b) {
6
7          return a + b;
8      }
9
10     public int substract(int a, int b) {
11
12         return a - 1;
13     }
14
15     public int multiply(int a, int b) {
16
17         return a * b;
18     }
19
20     public int divide(int a, int b) {
21
22         return a / b;
23     }
24
25
26 }
27
```

Method to be tested

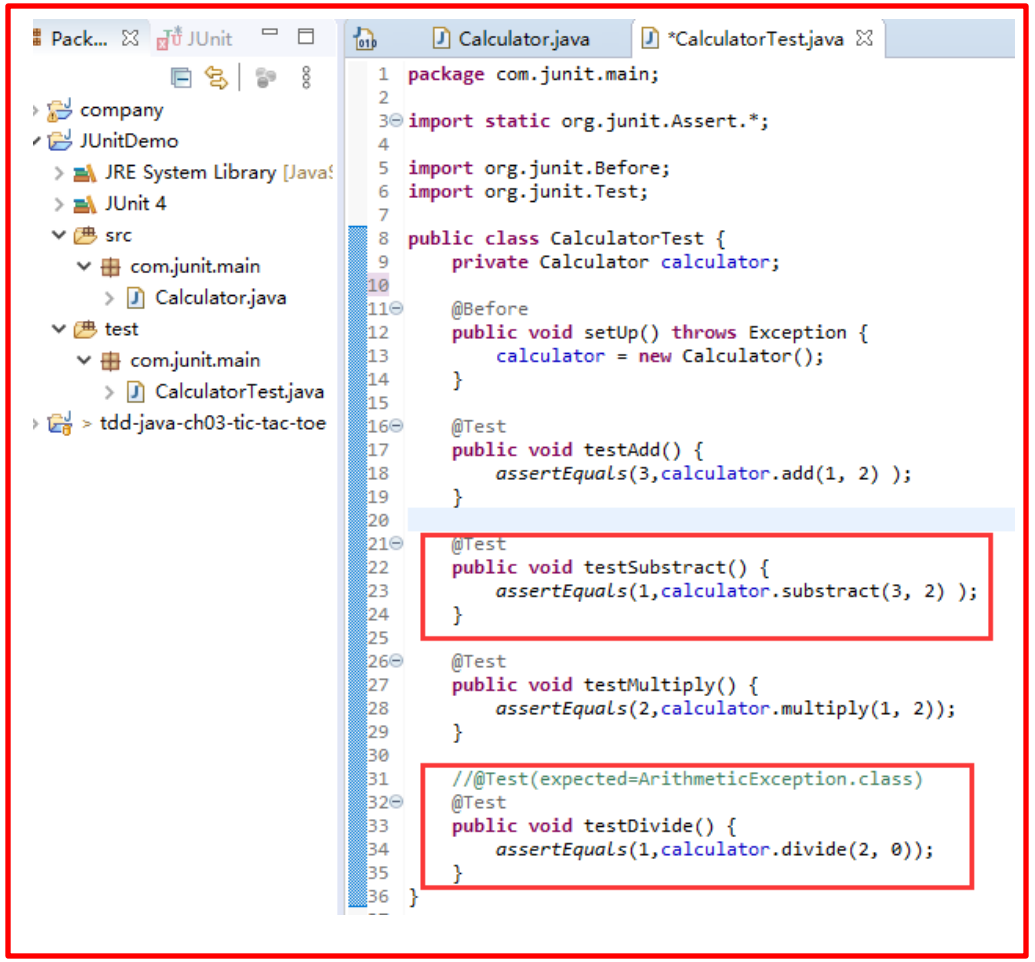
Calculator.java

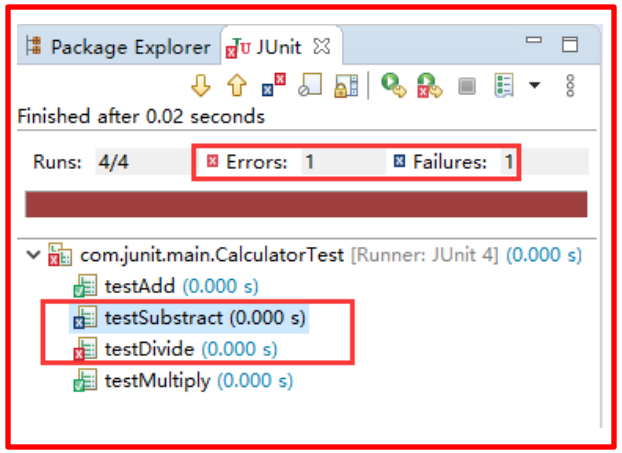
```
1  package com.junit.main;
2
3  public class Calculator {
4      public int add(int a, int b) {
5          return a + b;
6      }
7
8      public int substract(int a, int b) {
9          return a - 1;
10     }
11
12     public int multiply(int a, int b) {
13         return a * b;
14     }
15
16     public int divide(int a, int b) {
17             return a / b;
18     }
19
20     public static void main(String[] args)
21     {
22         Calculator calculator = new Calculator();
23         System.out.println(3 == calculator.add(1, 2));
24         System.out.println(1 == calculator.substract(3, 2));
25         System.out.println(2 == calculator.multiply(1, 2));
26         System.out.println(1 == calculator.divide(2, 2));
27     }
28
29 }
30
```

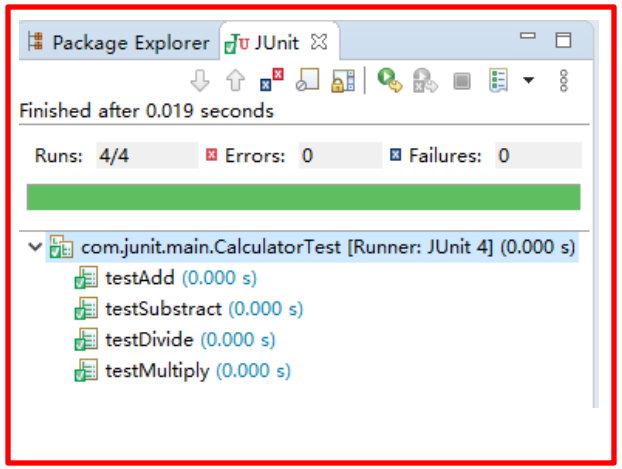
Problems Javadoc Declaration Console

```
<terminated> Calculator [Java Application] C:\Program Files\Java\jdk-11.0.2\bin\javaw
true
false
true
true
```

Unit tests and JUnit







Keeps the bar green to keeps the code clean!

——JUnit

Performance testing

Usually all the performance related to the test is collected and used by different people in different situations. Performance testing is a "normal" test that mainly tests whether the system meets the requirements when used, and may be slightly beyond the "normal" range to preserve the expansion space of the system.

Performance testing tool: Load impact
http://loadimpact.com/

Stress Testing

Stress testing is a type of performance test, which aims to test the stability of the system under a certain load for a long time, but this load is not necessarily caused by the application system itself. Stress testing focuses on changes in the performance of systems running for long periods of time under high volume (e.g., whether they are slow to react, whether the system gradually crashes due to memory leaks, and whether they can be recovered).

Stress testing tool: JMeter
Official website: https://jmeter.apache.org

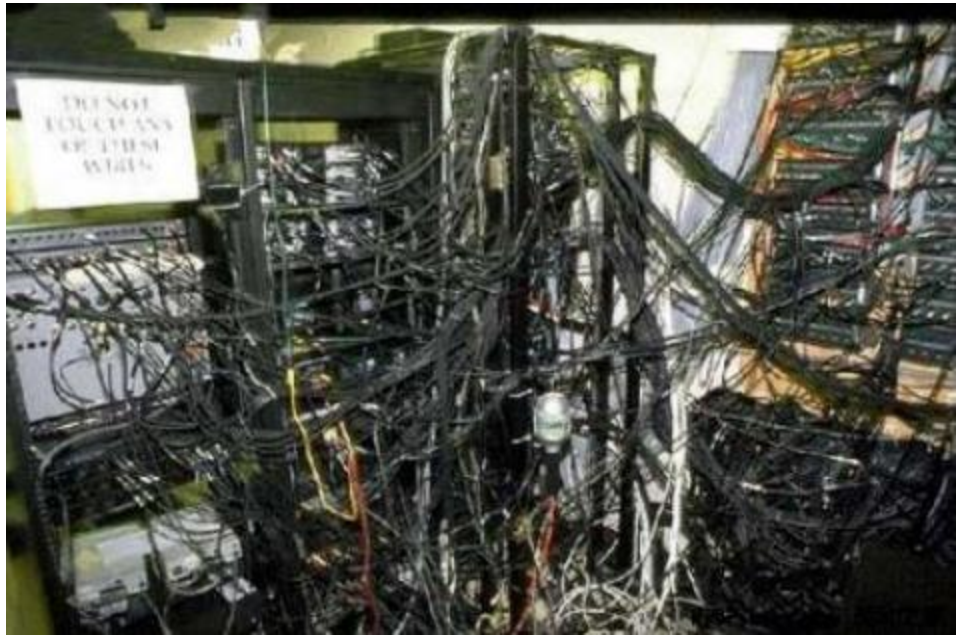
Code coverage testing

- **Code coverage = How much coverage a code has, a measure**
  - Statement Coverage
  - Decision Coverage
  - Condition Coverage
  - Path Coverage

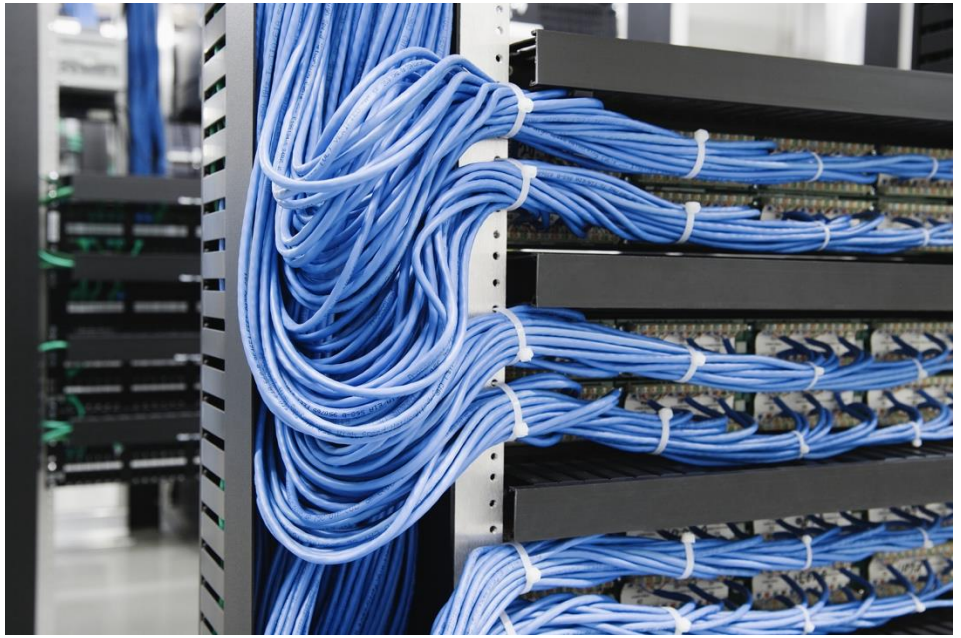
Code coverage testing tool: JaCoCo
official website: https://www.jacoco.org/jacoco/trunk/index.html

Code quality

Software quality is not only dependent on architecture and project management, but also closely related to code quality. Code quality is an integral part of software development.



Bad Code



Good Code

How to evaluate code quality?

1. **Maintainability**
   - Whether it is possible to quickly modify or add code without destroying the original code design or introducing new bugs.
2. **Readability**
   - Whether the code complies with the coding specifications, whether the naming is meaningful, whether the comments are detailed, whether the function is of appropriate length, whether the module division is clear, whether it conforms to high cohesion and low coupling, etc.
3. **Extensibility**
   - If the original code is not modified or slightly modified, can new functional code be added by extension?

How to evaluate code quality?

4. Reusability

- The number one bad smell in code is code duplication.

——Kent Benk and Martin Fowler

- Reusable, generic code should be written and existing code should be reused.

5. Testability

- whether it is easy to write unit tests against code;
- Poor code testability and low unit test coverage often mean that the code is not well designed.

6. Simplicity

- KISS principle: Keep It Simple, Stupid
- Try to keep the code simple and logical.

How to improve code quality

1. Strictly follow coding specifications

2. Write high-quality unit tests

3. Code review

4. Develop documents first

5. Continuous refactoring, refactoring, refactoring

Question

Which of the following descriptions of white box testing and black box testing is incorrect?

A. White box testing focuses on errors in software coding, while black box testing focuses on functional errors

B. White box testing is demanding on testers, while black box testing is relatively less demanding

C. White box tests have low code coverage, while black box tests have very high code coverage

D. White box testing focuses on the developer perspective, while black box testing focuses on the user perspective

submit

question

Which of the following descriptions of white box testing and black box testing is incorrect?

A. White box testing focuses on errors in software coding, while black box testing focuses on functional errors

B. White box testing is demanding on testers, while black box testing is relatively less demanding

C. White box tests have low code coverage, while black box tests have very high code coverage

D. White box testing focuses on the developer perspective, while black box testing focuses on the user perspective

submit

Summary

1. Definition and classification of software testing
2. Definition of test cases and their design methodology
3. White Box Testing (Definition, Control Flow Chart, Logical Overlay Method, Test Case Design)
4. Black Box Testing (Definition, Equivalent Class Division, Boundary Value Analysis, Scenario Method)
5. Stress test, performance test and code coverage test
6. Code quality assurance